

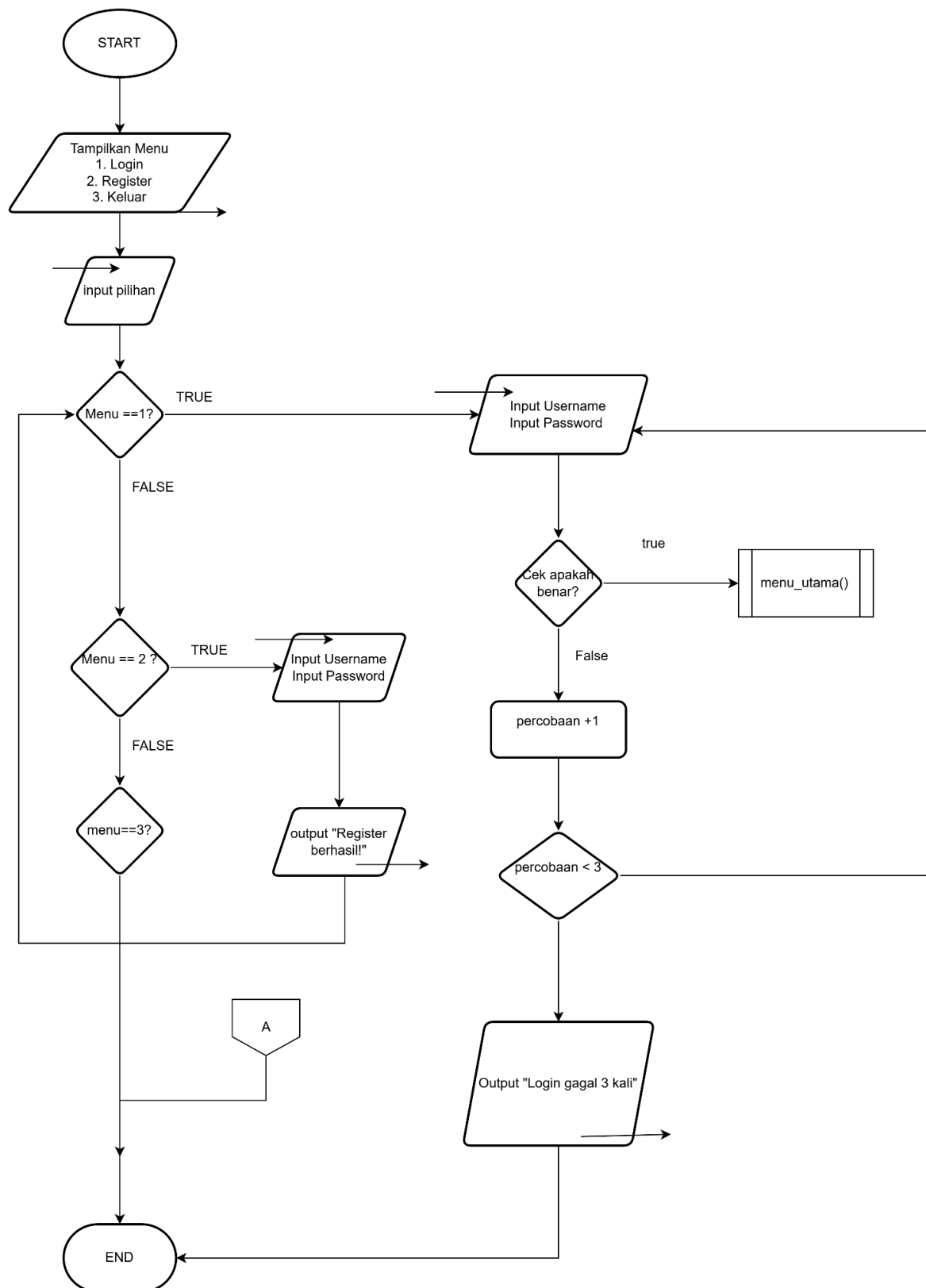
LAPORAN PRAKTIKUM
POSTTEST 7
ALGORITMA PEMROGRAMAN LANJUTAN



Disusun oleh:
Luvita Khairana Salwa (2509106041)
Kelas A'25

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

1. Flowchart



1.1 Menu Login

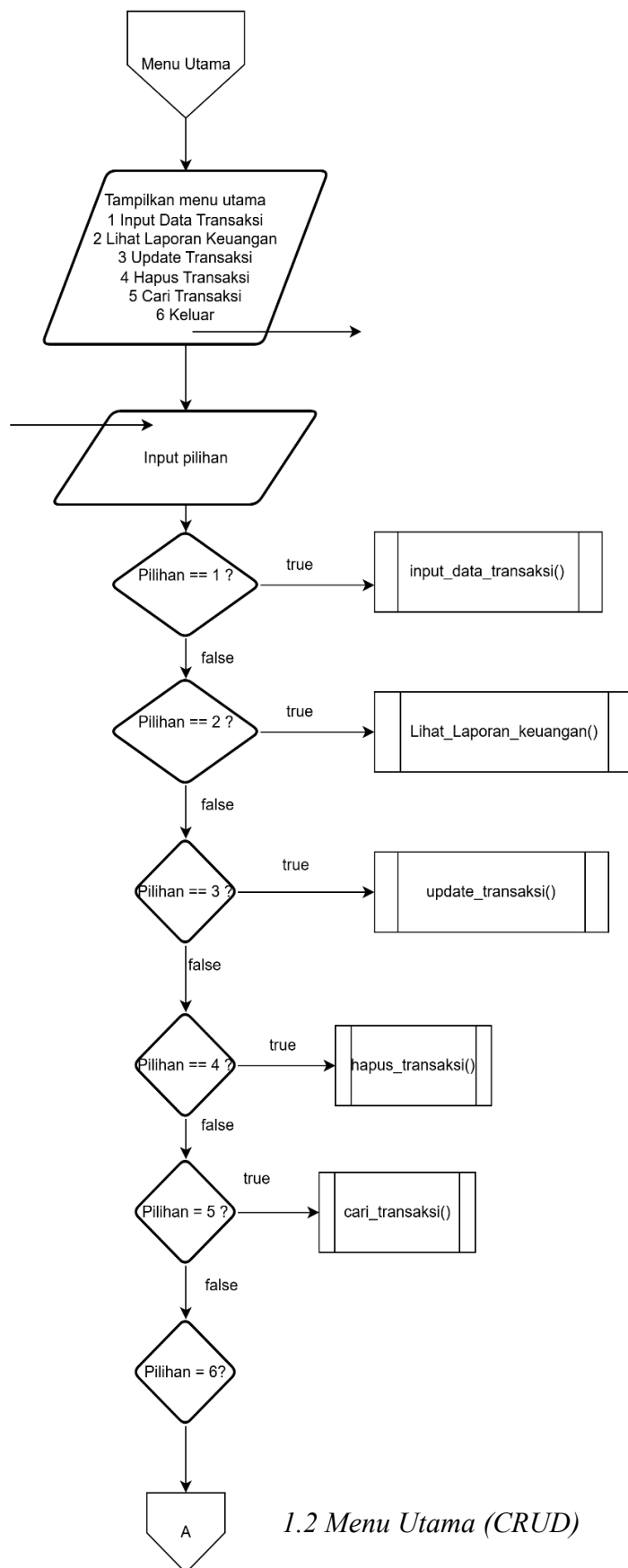
Penjelasan gambar 1.1 Menu login

Program dimulai dengan menampilkan menu yang berisi tiga pilihan yaitu Login, Register, dan Keluar. Pengguna kemudian diminta memasukkan pilihan menu sesuai dengan nomor yang tersedia. Jika pengguna memilih **Login**, sistem akan meminta pengguna untuk memasukkan username dan password. Setelah itu sistem akan memeriksa apakah username dan password yang dimasukkan sesuai dengan data yang tersimpan di dalam sistem. Jika data yang dimasukkan benar, maka pengguna berhasil login dan akan masuk ke menu utama.

Namun jika username atau password yang dimasukkan salah, pengguna masih diberikan kesempatan untuk mencoba login kembali. Proses login ini dibatasi maksimal tiga kali percobaan. Jika pengguna berhasil login sebelum tiga kali percobaan, maka pengguna dapat masuk ke menu utama. Tetapi jika pengguna gagal login sebanyak tiga kali, maka sistem akan langsung menghentikan program dan proses berakhir.

Jika pengguna memilih **Register**, sistem akan meminta pengguna untuk memasukkan username dan password baru. Data yang dimasukkan kemudian disimpan ke dalam sistem sebagai data akun baru. Setelah proses pendaftaran berhasil, pengguna akan langsung diarahkan kembali ke proses login untuk masuk menggunakan akun yang baru saja dibuat. Pada tahap login ini, pengguna juga hanya diberikan kesempatan maksimal tiga kali percobaan. Jika dalam tiga kali percobaan login masih gagal, maka program akan langsung berakhir.

Jika pengguna memilih **Keluar**, maka sistem akan langsung menghentikan program dan proses selesai tanpa menjalankan proses lainnya.



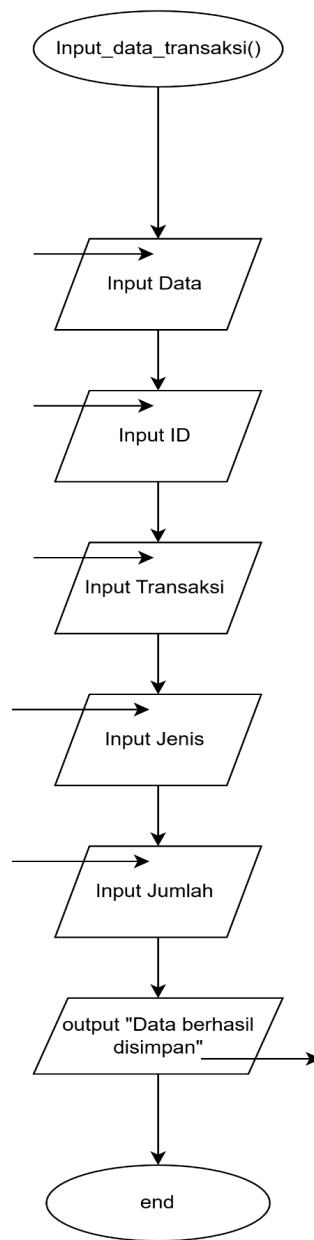
Penjelasan 1.2 Menu Utama (CRUD)

Setelah pengguna berhasil login, sistem menampilkan **Menu Utama** yang berisi beberapa pilihan yaitu Tambah transaksi, Lihat transaksi, Update transaksi, Hapus transaksi, c Cari transaksi dan Keluar. Pengguna kemudian memilih salah satu menu.

Jika memilih **Tambah Transaksi**, pengguna diminta memasukkan ID, nama transaksi, jenis transaksi (masuk atau keluar), dan jumlah uang. Data tersebut kemudian disimpan ke dalam sistem. Jika memilih **Lihat Transaksi**, sistem akan menampilkan seluruh data keuangan yang telah dimasukkan, serta menghitung total pemasukan, total pengeluaran, dan saldo akhir.

Jika memilih **Update Transaksi**, pengguna diminta memasukkan ID data yang ingin diubah. Jika ID ditemukan, pengguna dapat mengganti transaksi, jenis, dan jumlah data. Jika tidak ditemukan, sistem menampilkan pesan bahwa data tidak ditemukan. Jika memilih **Hapus Transaksi**, pengguna memasukkan ID data yang ingin dihapus. Jika data ditemukan, sistem akan menghapus data tersebut. Jika tidak ditemukan, sistem menampilkan pesan bahwa data tidak ditemukan.

Fitur **Cari Transaksi** berfungsi untuk menemukan data transaksi berdasarkan ID transaksi yang dimasukkan oleh pengguna. Proses pencarian dilakukan dengan memeriksa setiap data transaksi yang tersimpan pada sistem. Jika memilih **Keluar**, program berhenti dan proses selesai.

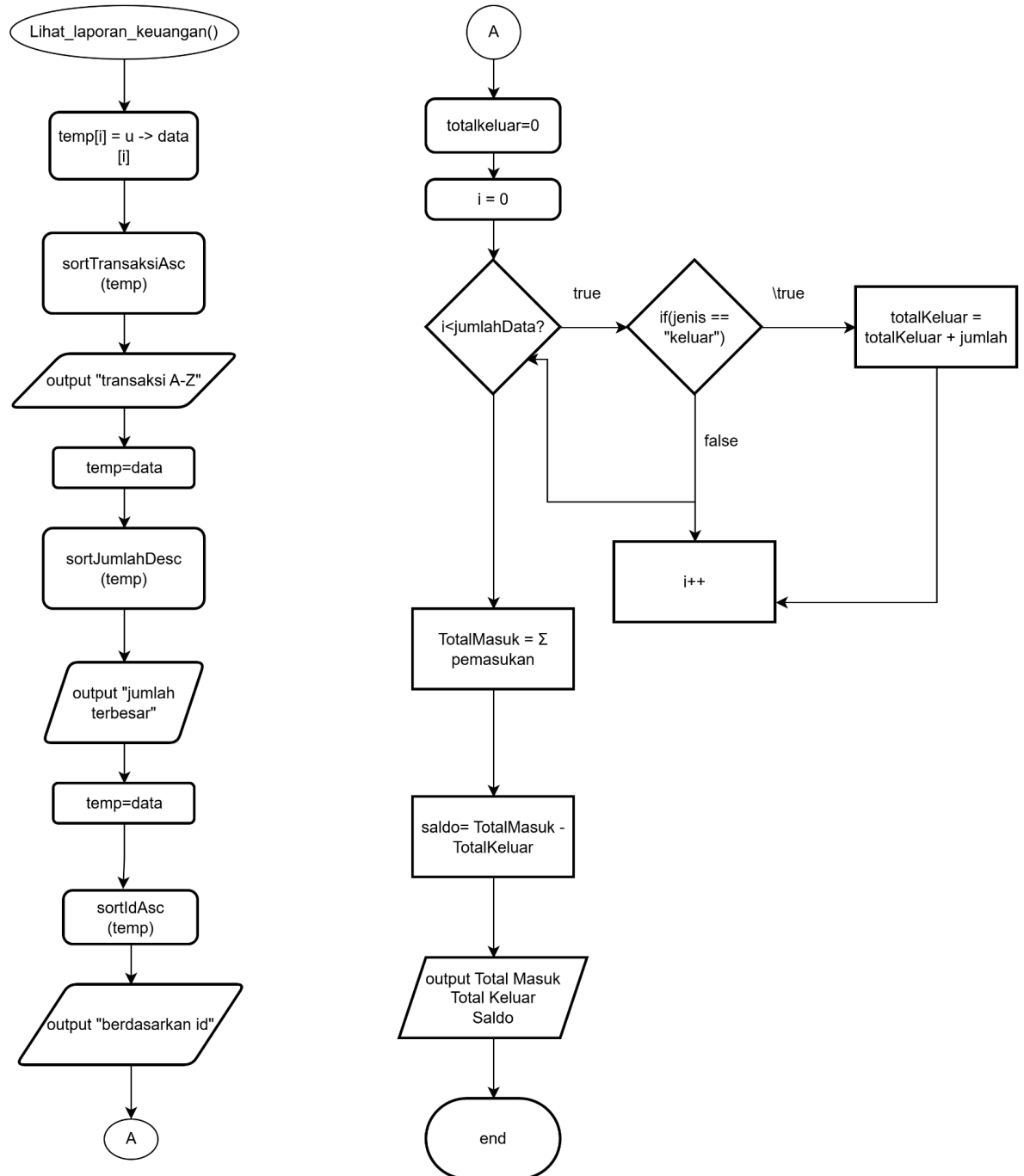


1.3 Flowchart tambah transaksi (create)

Penjelasan 1.3 Flowchart tambah data (Create)

Pada menu utama, jika pengguna memilih **Tambah Transaksi**, sistem akan meminta pengguna memasukkan ID transaksi. Setelah itu pengguna memasukkan nama transaksi, jenis transaksi (masuk atau keluar), dan jumlah uang.

Setelah semua data dimasukkan, sistem akan menyimpan data ke dalam array data keuangan. Kemudian jumlah data bertambah satu. Setelah data berhasil disimpan, sistem akan kembali ke Menu Utama untuk menunggu pilihan berikutnya dari pengguna.



1.4 Lihat Laporan Keuangan (Read)

Penjelasan 1.4 Lihat Laporan Keuangan (Read)

Mulai dari fungsi `lihat_laporan_keuangan()`. Langkah pertama adalah menyalin seluruh data dari `u->data` ke array sementara `temp`. Ini dilakukan supaya data asli tidak berubah saat proses sorting.

Setelah itu dilakukan sorting pertama menggunakan `sortTransaksiAsc(temp)`, yaitu mengurutkan data berdasarkan nama transaksi dari A ke Z. Hasilnya kemudian ditampilkan sebagai laporan “transaksi A-Z”. Selanjutnya data asli disalin lagi ke `temp`. Ini penting karena sebelumnya `temp` sudah berubah akibat sorting pertama. Setelah itu dilakukan sorting kedua dengan `sortJumlahDesc(temp)`, yaitu mengurutkan berdasarkan jumlah terbesar ke terkecil. Hasilnya ditampilkan sebagai laporan “jumlah terbesar”.

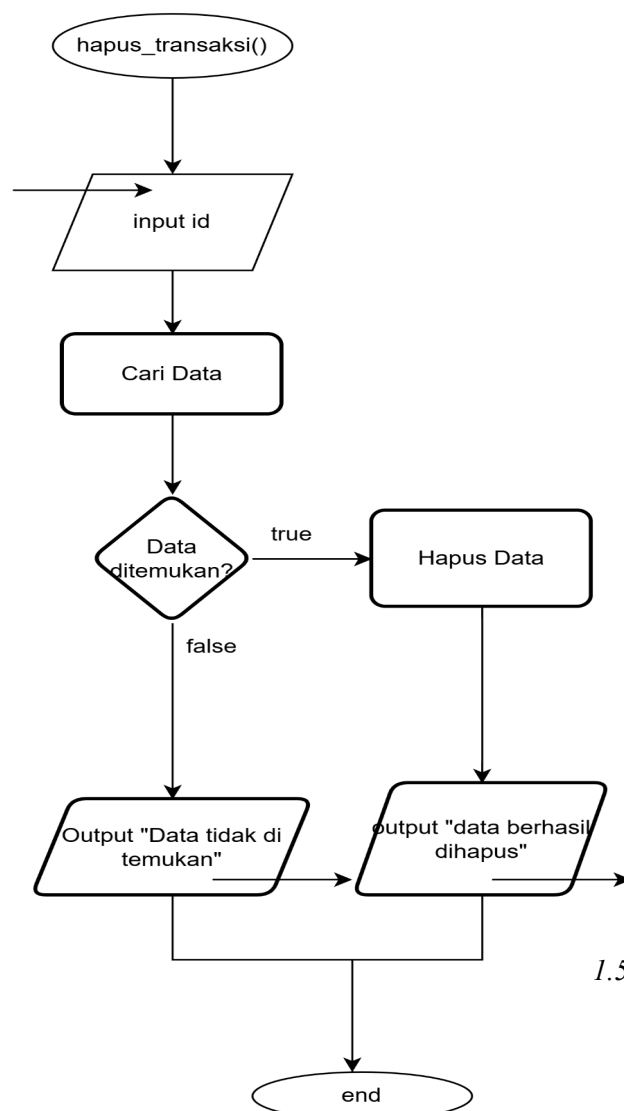
Kemudian data disalin lagi ke `temp` untuk ketiga kalinya. Dilakukan sorting menggunakan `sortIdAsc(temp)` untuk mengurutkan berdasarkan ID dari kecil ke besar. Hasilnya ditampilkan sebagai laporan “berdasarkan ID”. Setelah semua proses sorting dan output selesai, masuk ke bagian perhitungan total pengeluaran. Variabel `totalKeluar` diinisialisasi dengan 0, lalu dilakukan perulangan dari indeks 0 sampai jumlah data.

Di dalam perulangan terdapat kondisi:

jika `jenis == "keluar"` maka nilai jumlah akan ditambahkan ke `totalKeluar`. Jika tidak, data dilewati. Setiap iterasi, nilai indeks `i` bertambah sampai semua data diperiksa. Setelah itu dihitung `TotalMasuk` sebagai total seluruh pemasukan menggunakan fungsi `hitungTotalMasuk`. Kemudian dihitung saldo dengan rumus:

$$\text{saldo} = \text{TotalMasuk} - \text{TotalKeluar}.$$

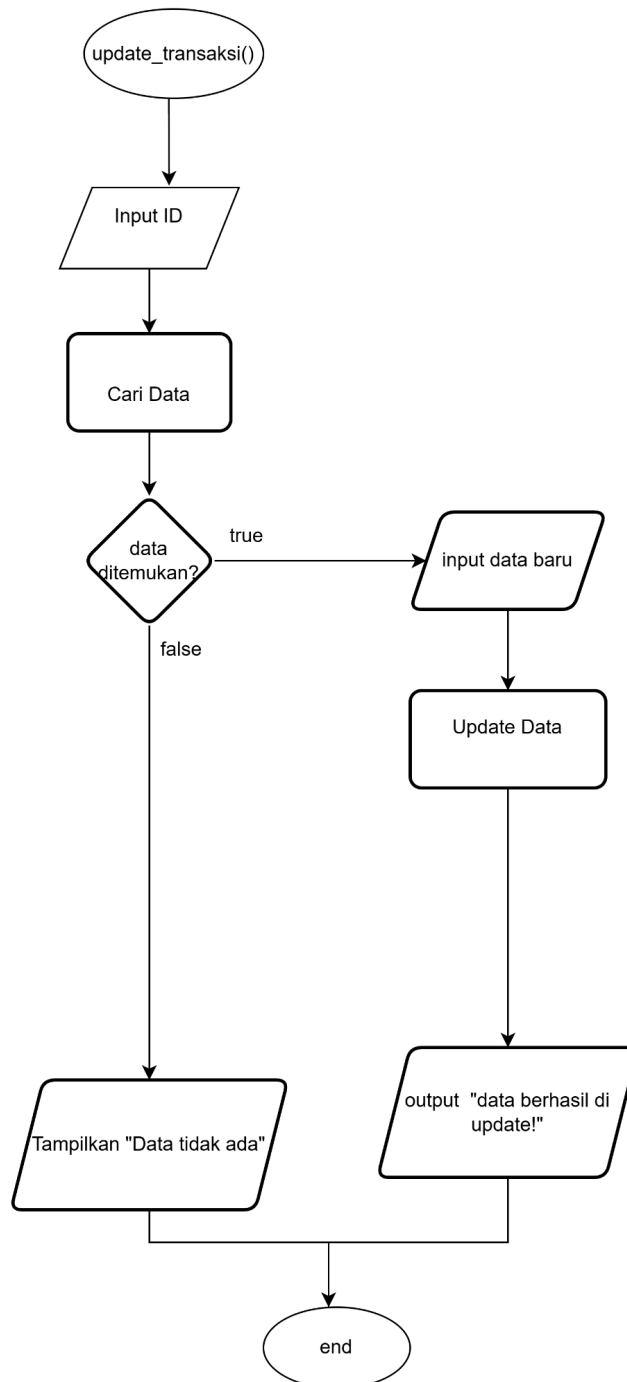
Langkah terakhir adalah menampilkan hasil akhir berupa Total Masuk, Total Keluar, dan Saldo. Proses berakhir di `end`.



1.5 Hapus transaksi (Delete)

Penjelasan 1.5 Hapus Transaksi (Delete)

Pada menu utama, jika pengguna memilih **Hapus Transaksi**, sistem akan meminta pengguna memasukkan ID data yang ingin dihapus. Sistem kemudian mencari ID tersebut pada data keuangan. Jika ID ditemukan, sistem akan menghapus data tersebut dari array dengan menggeser data setelahnya, lalu jumlah data dikurangi satu dan menampilkan pesan data berhasil dihapus. Jika ID tidak ditemukan, sistem akan menampilkan pesan data tidak ditemukan. Setelah proses selesai, program kembali ke Menu Utama.

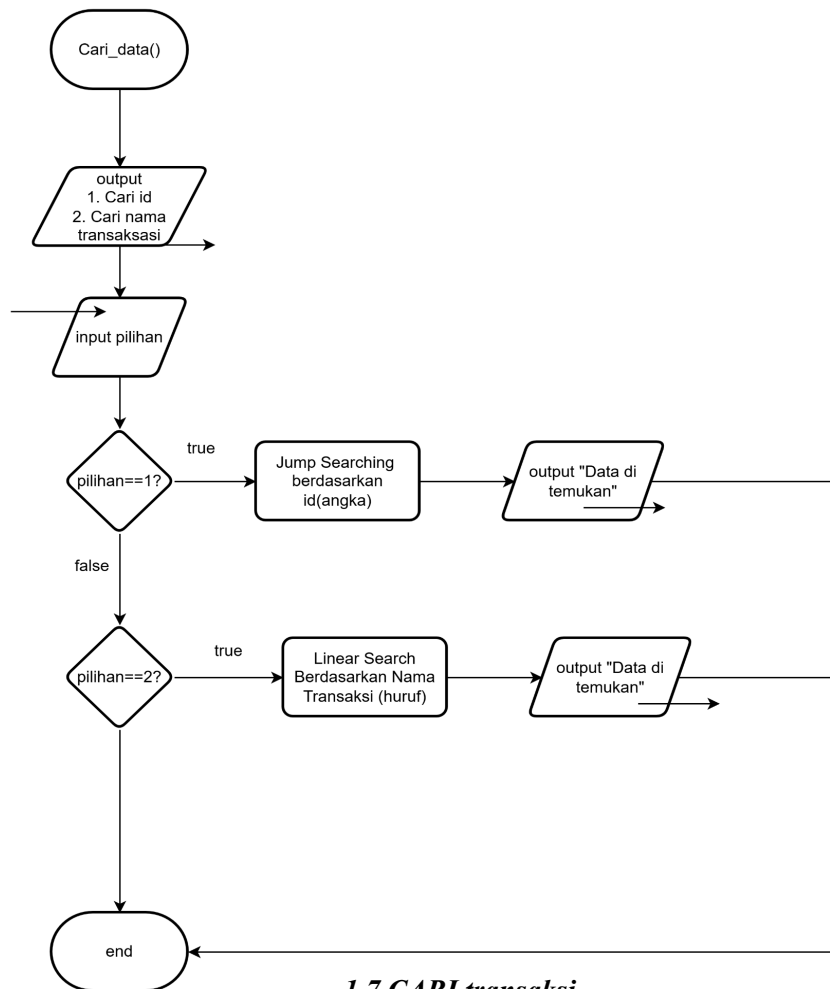


1.6 Update Transaksi

Penjelasan 1.6 Update Transaksi

Pada menu utama, jika pengguna memilih **Update Transaksi**, sistem akan meminta pengguna memasukkan ID data yang ingin diubah. Sistem kemudian mencari ID tersebut pada data keuangan. Jika ID ditemukan, sistem meminta pengguna memasukkan nama transaksi baru, jenis transaksi baru, dan jumlah baru. Setelah itu data akan diperbarui dan sistem menampilkan pesan data berhasil diupdate.

Jika ID tidak ditemukan, sistem akan menampilkan pesan data tidak ditemukan. Setelah proses selesai, program kembali ke Menu Utama.



1.7 CARI transaksi

Penjelasan 1.7 CARI transaksi

Fitur Cari Transaksi berfungsi untuk menemukan data transaksi berdasarkan ID transaksi yang dimasukkan oleh pengguna. Proses pencarian dilakukan dengan memeriksa setiap data transaksi yang tersimpan pada sistem.

Langkah pertama yang dilakukan adalah meminta pengguna untuk memasukkan ID transaksi yang ingin dicari. Setelah itu, sistem melakukan proses pencarian dengan menelusuri data transaksi satu per satu menggunakan perulangan.

Pada setiap data transaksi, sistem akan membandingkan ID transaksi yang dimasukkan dengan ID yang tersimpan pada data. Jika kedua ID tersebut sama, maka sistem akan menampilkan informasi transaksi yang terdiri dari ID transaksi, nama transaksi, jenis transaksi, dan jumlah transaksi.

Apabila ID yang dimasukkan tidak sesuai dengan data yang sedang diperiksa, sistem akan melanjutkan pencarian ke data berikutnya hingga seluruh data selesai diperiksa. Jika setelah proses pencarian selesai tidak ditemukan data dengan ID yang dimaksud, maka sistem akan menampilkan pesan bahwa data transaksi tidak ditemukan.

2. Deskripsi Singkat Program

Program ini merupakan Sistem Manajemen Keuangan Perusahaan berbasis bahasa pemrograman C++ yang dibuat untuk membantu pengguna dalam mengelola data transaksi keuangan. Program ini menggunakan konsep CRUD (Create, Read, Update, Delete) untuk menambah, menampilkan, mengubah, dan menghapus data transaksi.

Program diawali dengan tampilan menu awal yang terdiri dari tiga pilihan yaitu Login, Register, dan Keluar. Pada menu Login, pengguna dapat masuk ke dalam sistem menggunakan akun utama yang telah tersedia yaitu username vita dan password 041. Selain itu, pengguna juga dapat login menggunakan akun yang telah dibuat melalui menu register.

Pada menu Register, pengguna dapat membuat akun baru dengan memasukkan username dan password yang diinginkan. Akun tersebut kemudian dapat digunakan untuk login ke dalam sistem.

Pada proses login, pengguna diberikan maksimal tiga kali percobaan untuk memasukkan username dan password yang benar. Jika pengguna gagal melakukan login sebanyak tiga kali, maka program akan otomatis berhenti.

Setelah berhasil login, pengguna akan masuk ke menu utama sistem yang menampilkan beberapa pilihan menu yaitu menambah data transaksi, melihat data transaksi, mengubah data transaksi, menghapus data transaksi, dan keluar dari program. Program akan terus menampilkan menu ini secara berulang hingga pengguna memilih opsi keluar.

Data transaksi yang disimpan meliputi ID transaksi, tanggal transaksi, jenis transaksi (pemasukan atau pengeluaran), kategori transaksi, keterangan transaksi, dan nominal uang. Program ini menggunakan struct dan nested struct untuk menyimpan data secara terstruktur serta menggunakan array of struct untuk menyimpan banyak data transaksi.

Selain itu, program ini juga menerapkan konsep pointer untuk meningkatkan efisiensi dalam pengolahan data. Pointer digunakan untuk mengirim alamat data struct ke dalam fungsi menggunakan address-of operator (&), sehingga tidak perlu melakukan penyalinan data secara keseluruhan. Di dalam fungsi, data diakses menggunakan dereference operator (*) dan operator panah (->) untuk mengambil nilai dari alamat tersebut. Dengan penggunaan pointer ini, perubahan data yang dilakukan di dalam fungsi akan langsung mempengaruhi data asli, sehingga program menjadi lebih efisien dalam penggunaan memori dan pengolahan data.

3. Source Code

```
//nested crud
struct Keuangan{
    int id;
    string transaksi;
    string jenis;
    int jumlah;
};

struct User{
    string nama;
    string password;

    Keuangan data[100];
    int jumlahData;
};
```

Ada struct User dan Struct Keuangan yang berguna menyimpan data.

1. User → menyimpan username & password
2. Keuangan → menyimpan data transaksi

```
User users[10]; // max user
    int jumlahUser = 1;

    // akun admin awal
    users[0].nama = "Vita";
    users[0].password = "041";

    int menuAwal;
    string username, password;
    bool loginBerhasil = false;
```

```
int percobaan = 0;
while(percobaan < 3)
```

1. Login hanya **3 kali kesempatan**

```
if(!loginBerhasil){
    cout << "Kesempatan login habis.\n";
    return 0;
}
```

```
User users[10]; // max user
    int jumlahUser = 1;
    // akun admin awal
```

```

users[0].nama = "Vita";
users[0].password = "041";
int menuAwal;
string username, password;
bool loginBerhasil = false;

```

1. User users[10]; Membuat array bertipe User untuk menyimpan maksimal 10 akun pengguna.
2. int jumlahUser = 1; Variabel untuk menghitung jumlah user yang ada, dimulai dari 1 karena sudah ada akun awal.
3. users[0].nama = "Vita";
users[0].password = "041"; Menetapkan akun admin awal dengan username Vita dan password 041.
4. int menuAwal; Variabel untuk menyimpan pilihan menu awal (Login, Register, atau Keluar).
5. string username, password; Variabel untuk menyimpan input username dan password saat login.
6. bool loginBerhasil = false; Variabel boolean untuk mengecek apakah login berhasil atau tidak. Awalnya bernilai false (belum berhasil).

```

int totalMasuk = 0;
int totalKeluar = 0;

```

Membuat dua variabel untuk menyimpan total uang:

- 1) totalMasuk → total pemasukan
- 2) totalKeluar → total pengeluaran

Nilai awalnya 0.

```

for(int i = 0; i < jumlahData; i++){

```

Perulangan for digunakan untuk membaca semua data transaksi dari array data.

- 1) dimulai dari i = 0
- 2) sampai jumlahData

```

cout << " ID          : " << data[i].id << endl;
cout << " Transaksi : " << data[i].transaksi << endl;
cout << " Jenis      : " << data[i].jenis << endl;
cout << " Jumlah     : " << data[i].jumlah << endl;

```

Bagian ini menampilkan informasi transaksi:

- 1) ID transaksi

- 2) Nama transaksi
- 3) Jenis transaksi (masuk / keluar)
- 4) Jumlah uang

```
if(data[i].jenis == "masuk"){
    totalMasuk += data[i].jumlah;
}
else if(data[i].jenis == "keluar"){
    totalKeluar += data[i].jumlah;
}
```

Program mengecek jenis transaksi:

1. Jika masuk, jumlah uang ditambahkan ke totalMasuk
2. Jika keluar, jumlah uang ditambahkan ke totalKeluar

artinya ditambah dengan nilai sebelumnya.

```
totalMasuk = totalMasuk + data[i].jumlah
```

```
int saldo = totalMasuk - totalKeluar;
```

```
cout << " Total Pemasukan   : " << totalMasuk << endl;
cout << " Total Pengeluaran : " << totalKeluar << endl;
cout << " Saldo Akhir        : " << saldo << endl;
```

```
cin.ignore();
cin.get();

setw(20)
```

1. cin.ignore() membersihkan sisa input
2. cin.get() menunggu user menekan Enter

Biasanya dipakai di akhir menu supaya program tidak langsung kembali ke menu utama

Setw berarti set width (mengatur lebar kolom output). Jadi setw(20) artinya memberi ruang 20 karakter untuk data yang dicetak setelahnya.

```
#include <iostream>
#include <iomanip>
#include <limits>
```



```
#include <cstdlib>
using namespace std;
```

1. #include <iostream>

Digunakan untuk operasi input dan output standar, seperti cin (input dari keyboard) dan cout (menampilkan output ke layar).

2. #include <iomanip>

Digunakan untuk mengatur format tampilan output, misalnya mengatur jumlah angka di belakang koma atau merapikan tampilan data.

3. #include <limits>

Digunakan untuk mengetahui atau menggunakan batas nilai maksimum dan minimum dari suatu tipe data.

4. #include <cstdlib>

Berisi fungsi-fungsi umum dari pustaka standar C, seperti fungsi untuk angka acak (rand()), menghentikan program (exit()), atau menjalankan perintah sistem.

5. using namespace std;

Digunakan agar kita tidak perlu menuliskan std:: di depan fungsi atau objek dari library standar, seperti std::cout atau std::cin.

```
// REKURSIF
int hitungTotalMasuk(Kuangan data[], int n){

    if(n == 0)
        return 0;

    if(data[n-1].jenis == "masuk")
        return data[n-1].jumlah + hitungTotalMasuk(data, n-1);

    return hitungTotalMasuk(data, n-1);
}

//overloading
int hitungSaldo(int masuk, int keluar){
    return masuk - keluar;
}

int hitungSaldo(User u){

    int masuk = 0;
    int keluar = 0;

    for(int i=0;i<u.jumlahData;i++){

        if(u.data[i].jenis == "masuk")
            masuk += u.data[i].jumlah;
    }
}
```

```

        else if(u.data[i].jenis == "keluar")
            keluar += u.data[i].jumlah;
    }

    return masuk - keluar;
}

```

1. Fungsi Rekursif hitungTotalMasuk

Fungsi hitungTotalMasuk digunakan untuk menghitung total seluruh transaksi yang berjenis pemasukan (masuk). Fungsi ini menggunakan metode rekursif, yaitu fungsi yang memanggil dirinya sendiri sampai mencapai kondisi tertentu. Fungsi menerima dua parameter, yaitu array data yang berisi kumpulan transaksi dan variabel n yang menunjukkan jumlah data transaksi yang akan diperiksa.

Langkah kerja fungsi adalah sebagai berikut:

1. Jika nilai n sama dengan 0, maka fungsi mengembalikan nilai 0. Kondisi ini merupakan base case yang menandakan bahwa tidak ada lagi data yang perlu diperiksa.
2. Jika jenis transaksi pada data ke-(n-1) adalah "masuk", maka jumlah transaksi tersebut akan ditambahkan dengan hasil pemanggilan fungsi hitungTotalMasuk pada data sebelumnya.
3. Jika jenis transaksi bukan "masuk", maka fungsi akan langsung memanggil kembali hitungTotalMasuk untuk memeriksa data sebelumnya tanpa menambahkan nilai transaksi tersebut.

Dengan cara ini, fungsi akan memeriksa seluruh data transaksi satu per satu hingga semua data selesai dihitung.

2. Fungsi Overloading hitungSaldo

Program menggunakan konsep overloading fungsi, yaitu penggunaan nama fungsi yang sama tetapi dengan parameter yang berbeda. Fungsi pertama hitungSaldo(int masuk, int keluar) digunakan untuk menghitung saldo dengan cara mengurangkan total pemasukan dengan total pengeluaran. Fungsi ini menerima dua parameter berupa jumlah pemasukan dan jumlah pengeluaran, kemudian mengembalikan hasil selisih keduanya.

Fungsi kedua hitungSaldo(User u) menerima parameter berupa struktur User. Pada fungsi ini dilakukan proses perhitungan total pemasukan dan total pengeluaran secara langsung dari data transaksi yang dimiliki oleh pengguna.

Langkah kerja fungsi tersebut adalah:

1. Membuat dua variabel yaitu masuk dan keluar dengan nilai awal 0.
2. Melakukan perulangan untuk memeriksa seluruh data transaksi yang dimiliki pengguna.
3. jenis transaksi adalah "masuk", maka nilai transaksi akan ditambahkan ke variabel masuk.
4. Jika jenis transaksi adalah "keluar", maka nilai transaksi akan ditambahkan ke variabel keluar.
5. Setelah seluruh data diperiksa, fungsi akan mengembalikan hasil pengurangan antara total pemasukan dan total pengeluaran sebagai nilai saldo.

Dengan adanya fungsi overloading ini, perhitungan saldo dapat dilakukan dengan dua cara yang berbeda, yaitu menggunakan total pemasukan dan pengeluaran yang sudah diketahui, atau langsung dari seluruh data transaksi yang dimiliki oleh pengguna.

```
users[userLogin].data[i].id = i + 1;
```

Baris program ini dinamakan id autogenerate dimana memberikan ID transaksi secara otomatis berdasarkan urutan data yang dimasukkan.

A. INPUT DATA

```
inputTransaksi(User *u)
```

Fungsi untuk menambah transaksi:

1. Cek apakah data penuh
2. Input nama transaksi
3. Input jenis (masuk/keluar)
4. Input jumlah
5. Simpan ke array

Menggunakan pointer:

1. `u->data[i]` untuk akses data

B. LIHAT LAPORAN

```
lihatLaporan(User *u)
```

Fungsi untuk menampilkan semua transaksi:

1. Menampilkan tabel
2. Menghitung total keluar (loop)
3. Menghitung total masuk (rekursif)
4. Menghitung saldo

C. UPDATE

`updateTransaksi(User *u)`

Fungsi untuk mengubah data:

1. Cari berdasarkan ID
2. Jika ditemukan → update isi data

D. HAPUS

`hapusTransaksi(User *u)`

Fungsi untuk menghapus data:

1. Cari ID
2. Geser array ke kiri
3. Kurangi jumlahData

E. CARI

```
void cariData(User *u)
{
    int pilih;
    bool ketemu=false;
    garis();
    cout<<"          PENCARIAN TRANSAKSI\n";
    garis();
    cout<<"1. Cari ID \n";
    cout<<"2. Cari Nama Transaksi \n";
    cout<<"Pilih : ";
    cin>>pilih;
    cin.ignore(numeric_limits<streamsize>::max(),'\n');
    if(pilih==1){
        int cari;
        cout<<"Masukkan ID transaksi : ";
        cin>>cari;
        cin.ignore(numeric_limits<streamsize>::max(),'\n');

        sortIdAsc(u->data, u->jumlahData); // di sorting dulu egiluy
        int index = jumpSearchID(u->data, u->jumlahData, cari);
        if(index != -1){
            cout<<"\n>>> Data ditemukan <<<\n";
            cout<<"ID          : "<<u->data[index].id<<endl;
            cout<<"Transaksi : "<<u->data[index].transaksi<<endl;
            cout<<"Jenis      : "<<u->data[index].jenis<<endl;
            cout<<"Jumlah     : "<<u->data[index].jumlah<<endl;
        }else{
            cout<<"Data tidak ditemukan\n";
        }
    }
    // Linear search
    else if(pilih==2){
```

```

        string cari;
        cout<<"Masukkan nama transaksi : ";
        getline(cin,cari);
        for(int i=0;i<u->jumlahData;i++){
            if(u->data[i].transaksi == cari){
                cout<<"\n>>> Data ditemukan <<<\n";
                cout<<"ID          : "<<u->data[i].id<<endl;
                cout<<"Transaksi : "<<u->data[i].transaksi<<endl;
                cout<<"Jenis      : "<<u->data[i].jenis<<endl;
                cout<<"Jumlah     : "<<u->data[i].jumlah<<endl;
                ketemu=true;
            }
        }
    }
}

```

Fungsi cariData(User *u) digunakan untuk mencari data transaksi dengan dua metode berbeda, tergantung pilihan user.

Program pertama-tama menampilkan judul dan pilihan metode pencarian, yaitu berdasarkan ID atau berdasarkan nama transaksi. Input pilihan disimpan dalam variabel pilih.

Jika user memilih opsi 1, program akan melakukan pencarian berdasarkan ID menggunakan metode Jump Search. User diminta memasukkan ID yang ingin dicari, kemudian data terlebih dahulu diurutkan berdasarkan ID menggunakan fungsi sortIdAsc. Pengurutan ini wajib karena Jump Search hanya bisa digunakan pada data yang sudah terurut. Setelah data terurut, fungsi jumpSearchID dipanggil untuk mencari posisi data. Jika data ditemukan (index tidak -1), maka program menampilkan seluruh informasi transaksi seperti ID, nama transaksi, jenis, dan jumlah. Jika tidak ditemukan, program menampilkan pesan bahwa data tidak ada.

Jika user memilih opsi 2, program menggunakan Linear Search untuk mencari berdasarkan nama transaksi. User diminta memasukkan nama transaksi, kemudian program melakukan perulangan dari awal sampai akhir data. Setiap data dibandingkan dengan input. Jika ada yang cocok, data tersebut ditampilkan. Berbeda dengan Jump Search, metode ini tidak membutuhkan data dalam keadaan terurut.

Variabel ketemu digunakan untuk menandai apakah data ditemukan atau tidak pada pencarian teks. Jika tidak ada data yang cocok, nantinya bisa digunakan untuk menampilkan pesan bahwa data tidak ditemukan.

F. MENU UTAMA

```
menuUtama(User *user)
```

Pusat kontrol program:

1. Menampilkan menu
2. Memanggil fungsi sesuai pilihan user
3. Loop sampai pilih keluar

POINTER DIPAKAI:

1. parameter → `User *user`
2. akses → `user->data`

G. MAIN (PROGRAM UTAMA)

Fungsi utama program:

1. Menyimpan data user
2. Menampilkan menu login/register
3. Validasi login
4. Jika berhasil → masuk ke menu utama

H. POINTER PENTING:

`menuUtama(&users[userLogin]);` Artinya mengirim alamat data user ke fungsi.

`&` = address-of operator

`&users[userLogin]` artinya → mengambil alamat memori dari user yang login

Alamat ini dikirim ke fungsi `menuUtama(User *user)`

```
User *user
user->nama
user->data[i]
```

Struct pointer adalah pointer yang menunjuk ke data bertipe struct. Pada program ini digunakan untuk mengirim alamat data user ke fungsi menggunakan address-of operator, dan diakses menggunakan dereference operator dengan tanda panah (`->`).

i. Lihat_laporan()

```
void lihatLaporan(User *u){

    Keuangan temp[100];
    int totalKeluar=0;

    // SORT TRANSAKSI A-Z
    for(int i=0;i<u->jumlahData;i++) temp[i]=u->data[i];
    sortTransaksiAsc(temp, u->jumlahData);

    garis();
    cout<<"    LAPORAN (SORT TRANSAKSI A-Z)\n";
    garis();

    for(int i=0;i<u->jumlahData;i++){
        cout<<temp[i].id<<" | "
```

```

        <<temp[i].transaksi<<" | "
        <<temp[i].jenis<<" | "
        <<temp[i].jumlah<<endl;
    }

    // SORT JUMLAH DESC
    for(int i=0;i<u->jumlahData;i++) temp[i]=u->data[i];
    sortJumlahDesc(temp, u->jumlahData);

    garis();
    cout<<"    LAPORAN (SORT JUMLAH TERBESAR)\n";
    garis();

    for(int i=0;i<u->jumlahData;i++){
        cout<<temp[i].id<<" | "
            <<temp[i].transaksi<<" | "
            <<temp[i].jenis<<" | "
            <<temp[i].jumlah<<endl;
    }

    // SORT ID ASC
    for(int i=0;i<u->jumlahData;i++) temp[i]=u->data[i];
    sortIdAsc(temp, u->jumlahData);

    garis();
    cout<<"    LAPORAN (SORT ID TERKECIL)\n";
    garis();

    for(int i=0;i<u->jumlahData;i++){
        cout<<temp[i].id<<" | "
            <<temp[i].transaksi<<" | "
            <<temp[i].jenis<<" | "
            <<temp[i].jumlah<<endl;
    }

    // TOTAL
    for(int i=0;i<u->jumlahData;i++){
        if(u->data[i].jenis=="keluar")
            totalKeluar+=u->data[i].jumlah;
    }

    int totalMasuk=hitungTotalMasuk(u->data,u->jumlahData);
    int saldo=hitungSaldo(totalMasuk,totalKeluar);

    garis();
    cout<<"Total Pemasukan    : "<<totalMasuk<<endl;
    cout<<"Total Pengeluaran  : "<<totalKeluar<<endl;
    cout<<"Saldo Perusahaan   : "<<saldo<<endl;

```

```

    garis();
}

```

Pertama, dibuat array sementara temp[100] dan variabel totalKeluar diinisialisasi dengan 0. Array temp digunakan agar proses sorting tidak mengubah data asli. Selanjutnya masuk ke proses pertama, yaitu menyalin semua data dari u->data ke temp. Setelah itu, data di temp diurutkan menggunakan fungsi sortTransaksiAsc, yaitu berdasarkan nama transaksi dari A ke Z. Setelah diurutkan, program menampilkan judul laporan lalu mencetak seluruh isi data satu per satu.

Kemudian dilakukan proses kedua. Data asli kembali disalin ke temp, lalu diurutkan menggunakan sortJumlahDesc, yaitu berdasarkan jumlah terbesar ke terkecil. Setelah itu ditampilkan kembali dalam bentuk laporan. Proses ketiga sama seperti sebelumnya. Data disalin lagi ke temp, kemudian diurutkan menggunakan sortIdAsc, yaitu berdasarkan ID dari yang terkecil ke terbesar. Hasilnya juga ditampilkan ke layar.

Setelah semua proses sorting selesai, program masuk ke bagian perhitungan total pengeluaran. Dilakukan perulangan untuk seluruh data. Jika ditemukan data dengan jenis == "keluar", maka nilai jumlah akan ditambahkan ke variabel totalKeluar. Setelah itu, dihitung total pemasukan menggunakan fungsi hitungTotalMasuk, dengan parameter data dan jumlah data. Kemudian dihitung saldo menggunakan fungsi hitungSaldo, yaitu selisih antara total pemasukan dan total pengeluaran. Terakhir, program menampilkan ringkasan berupa total pemasukan, total pengeluaran, dan saldo perusahaan, lalu fungsi selesai.

```

try {
    cout << "Masukkan ID transaksi : ";
    if (!(cin >> cari)) throw "ID harus angka!";
}
catch (const char* msg) {
    cout << msg << endl;
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    return;
}
cin.ignore(numeric_limits<streamsize>::max(), '\n');

sortIdAsc(u->data, u->jumlahData);
int index = jumpSearchID(u->data, u->jumlahData, cari);

if (index != -1) {
    cout << "\n>>> Data ditemukan <<<\n";
    cout << "ID          : " << u->data[index].id << endl;
    cout << "Transaksi : " << u->data[index].transaksi << endl;
    cout << "Jenis      : " << u->data[index].jenis << endl;
    cout << "Jumlah     : " << u->data[index].jumlah << endl;
}

```



```
ketemu = true;
```

Program meminta input ID dari user. Proses ini dibungkus dengan try-catch untuk memastikan input yang dimasukkan adalah angka. Jika user memasukkan selain angka, maka kondisi `!(cin >> cari)` akan bernilai true dan program melempar error dengan `throw "ID harus angka!"`. Error ini kemudian ditangkap di blok catch, ditampilkan ke layar, lalu dilakukan `cin.clear()` untuk menghapus status error dan `cin.ignore(...)` untuk membersihkan buffer input. Setelah itu fungsi langsung dihentikan dengan `return`, sehingga proses tidak dilanjutkan.

Jika input valid, program tetap melakukan `cin.ignore(...)` sekali lagi untuk memastikan tidak ada sisa karakter di buffer yang bisa mengganggu input berikutnya.

Selanjutnya, data transaksi diurutkan terlebih dahulu menggunakan `sortIdAsc`. Pengurutan ini penting karena algoritma pencarian yang digunakan, yaitu jump search, hanya bisa bekerja dengan benar jika data dalam keadaan terurut.

Setelah data terurut, program memanggil fungsi `jumpSearchID` untuk mencari ID yang dimasukkan. Fungsi ini akan mengembalikan index data jika ditemukan, atau -1 jika tidak ditemukan.

Jika index tidak sama dengan -1, berarti data ditemukan. Program kemudian menampilkan detail transaksi seperti ID, nama transaksi, jenis, dan jumlah. Terakhir, variabel `ketemu` diubah menjadi true sebagai penanda bahwa pencarian berhasil

```
#include <algorithm>
#include <windows.h>
```

`#include <windows.h>`: Memanggil library sistem Windows.

`Sleep(300)`: Fungsi dari library eksternal untuk memberikan jeda waktu (animasi).

`system("cls")`: Mengakses perintah sistem operasi untuk membersihkan layar.

`system("pause")`: Menahan layar agar tidak langsung tertutup.

Pewarnaan Teks (ANSI Color Codes):

- `\033[1;32m` : Mengubah teks menjadi Hijau Bold (untuk status "Berhasil").
- `\033[1;31m` : Mengubah teks menjadi Merah Bold (untuk status "Gagal/Error").
- `\033[1;36m` : Mengubah teks menjadi Cyan/Biru Muda (untuk nama User).

4. Hasil output

```
=====
                        SISTEM MANAJEMEN KEUANGAN
=====
1. Login
2. Register
3. Keluar
=====
Pilih menu : 2

===== REGISTER =====
Username : miaw
Password : 055
Register berhasil! Silakan login.

===== LOGIN =====
Username : miaw
Password : 055
```

4.1 Menu login®ister

```
Login berhasil!

=====
                        MENU UTAMA
=====
User : kucil
-----
1. Input Data Transaksi
2. Lihat Laporan Keuangan
3. Update Transaksi
4. Hapus Transaksi
5. Cari Transaksi
6. Keluar
=====
Pilih Menu : 
```

4.2 Menu Utama

```
=====
Pilih Menu : 2

=====
                        LAPORAN KEUANGAN
=====
No  ID      Transaksi      Jenis      Jumlah
-----
1   1       makanan       keluar     40000000
2   2       gaji          masuk      3000000
-----
Total Pemasukan   : 3000000
Total Pengeluaran : 40000000
Saldo Perusahaan  : -37000000
```

```
Pilih Menu : 1
```

```
===== INPUT DATA TRANSAKSI =====
```

```
ID Transaksi : 002
```

```
Nama Transaksi : gaji
```

```
Jenis (masuk/keluar) : masuk
```

```
Jumlah : 3000000
```

```
Data berhasil ditambahkan
```

4.4 Tambah Data (CREATE)

```
Pilih Menu : 3
```

```
Masukkan ID yang diupdate : 1
```

```
Transaksi baru : uang
```

```
Jenis baru : masuk
```

```
Jumlah baru : 1000000
```

```
Data berhasil diupdate
```

4.5 Ubah Data (UPDATE)

```
Pilih Menu : 4
```

```
Masukkan ID yang dihapus : 1
```

```
Data berhasil dihapus
```

4.6 Hapus Data (DELETE)

```
=====
```

```
PENCARIAN DATA TRANSAKSI
```

```
=====
```

```
Masukkan ID transaksi : 2
```

```
Data ditemukan
```

```
ID : 2
```

```
Transaksi : gaji
```

```
Jenis : masuk
```

```
Jumlah : 3000000
```

4.7 Pencarian data

```
Pilih Menu : 6
```

```
=====
```

```
PROGRAM SELESAI DIGUNAKAN
```

```
=====
```

4.8 Keluar

```
Pilih : 1
Masukkan ID transaksi : 1

>>> Data ditemukan <<<
ID      : 1
Transaksi :
Jenis    : masuk
Jumlah   : 2000000
```

4.9 Cari Data (Jump Search)

```
=====
Pilih menu : 1
Masukkan ID transaksi : 1

>>> Data ditemukan <<<
ID      : 1
Transaksi : makanan
Jenis    : masuk
Jumlah   : 21111
=====
```

4.10 Cari Data (Linear Search)

5. Langkah-langkah GIT

```
PS C:\Users\miaw miaw\OneDrive\Dokumen\GIT-DIR-APL> git add ./pertemuan-2
fatal: pathspec './pertemuan-2' did not match any files
PS C:\Users\miaw miaw\OneDrive\Dokumen\GIT-DIR-APL> git add ./kelas
PS C:\Users\miaw miaw\OneDrive\Dokumen\GIT-DIR-APL> git commit -m "pertemuan2"
[main dc8fdda] pertemuan2
1 file changed, 94 insertions(+)
create mode 100644 kelas/pertemuan-2/pertemuan-2.cpp
PS C:\Users\miaw miaw\OneDrive\Dokumen\GIT-DIR-APL> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 1.17 KiB | 300.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/miawicey/APL.git
b059d12..dc8fdda main -> main
```

5.0.1 Langkah langkah git

5.1 GIT Add

Git add itu fungsi nya menambah file. Sebenarnya git add “nama file” tapi bisa “git add .” jika ingin menambah file keseluruhan

```
git add ./pertemuan-2
files
```

5.1.1 git add

5.2 GIT Commit

Berfungsi menyimpan file yang udah di add ke repo tadi kayak biar hasil yang udah di add, menjadi file permanen di repo git

git commit -m “perintah”

```
L> git add ./kelas
L> git commit -m "pertemuan2"
```

5.2.1 Git commit

5.3 GIT Push

Mengirim commit dari repo lokal ke repo github jadi yaaaa reponya akan muncul secara online dan dapat di lihat di repostory github

“Git push -u origin main”

```
PS C:\Users\miaw miaw\OneDrive\Dokumen\GIT-DIR-APL> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 1.17 KiB | 300.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/miawicou/APL.git
```

5.3.1 Git push